HSLU Lucerne University of Applied Sciences and Arts

School of Computer Science and Information Technology
Lucerne University of Applied Sciences and Arts (Switzerland)

Energy Data Analytics & Forecasting

Photovoltaic forecasting

by

Flora Gashi
Teckla Achieng Onyango
Sevan Sherbetjian

Contents

List of Figures

List of Tables

Chapter 1

Problem definition

1.1 Photovoltaic (PV) Power

Photovoltaic (PV) systems convert solar radiation into electrical energy and have become one of the fastest-growing renewable technologies due to their scalability, declining costs, and low environmental impact [1]. However, PV generation is inherently variable because it depends on meteorological conditions such as irradiance, cloud cover, and temperature, which creates challenges for grid stability and balancing supply and demand.

Accurate forecasting is therefore essential for efficient grid integration, as PV systems cannot be dispatched on demand like conventional power plants [2]. PV forecasting supports operational decisions such as energy trading, dispatch planning, reserve allocation, and storage management by reducing uncertainty and improving system reliability [3].

1.2 Machine Learning for PV Power Forecasting

To address the uncertainty and intermittency of PV output, accurate forecasting has become an essential operational requirement for modern power systems. PV power forecasting involves predicting the amount of electricity a solar system will generate over a specific future horizon, which can range from minutes ahead to day-ahead and seasonal forecasts, with day-ahead forecasting being particularly important for electricity market operations and scheduling [2, 4].

Generally, forecasting techniques fall into three primary categories: physical, statistical, and machine learning. Physical models depend on complex numerical weather predictions and detailed system specifications. In contrast, statistical and machine learning approaches derive insights directly from historical data, learning the correlation between weather variables and power output. Recently, machine learning has driven a significant leap in forecast precision by effectively mapping non-linear relationships and temporal dependencies.

A typical PV forecasting workflow involves collecting generation data from the PV system alongside meteorological measurements, which are then transmitted to a monitoring platform where forecasting models are applied [1]. The monitoring

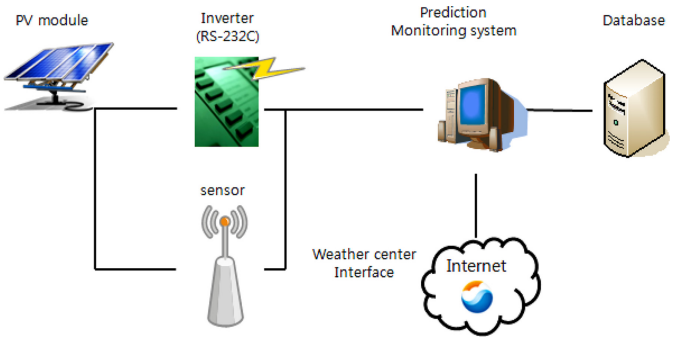

Figure 1.1: Network and hardware configuration of a PV monitoring system [1]

systems integrate sensors, inverters, and weather interfaces to continuously gather data that supports prediction and analysis.

1.3 Project Description

This project focuses on developing a photovoltaic (PV) power forecasting model for the local energy community Am Aawasser in Buochs, Switzerland with a goal of producing a day-ahead forecast of PV power generation for the next 24 hours using historical operational and weather data from the site.

Three datasets are provided, the PV generation data at 15-minute resolution, the household load data providing context on local demand, and weather data including irradiance, temperature, humidity, and cloud cover at hour intervals.

Following the project guidelines, the emphasis was placed on methodological soundness rather than achieving the lowest possible error. Particular attention was given to robust preprocessing, preventing data leakage, chronological time-series splitting, and transparent evaluation. The workflow included exploratory data analysis, preprocessing and temporal alignment, feature engineering, and the development and validation of forecasting models.

Chapter 2

Literature Review on Forecasting Models

Over the past decade, forecasting approaches have evolved from simple statistical models to advanced machine learning and deep learning architectures. This section reviews five major model categories commonly used in PV forecasting: Linear Models, Tree-Based Machine Learning Models, Long Short-Term Memory (LSTM), Temporal Convolutional Networks (TCN) and Gated Recurrent Unit (GRU).

2.1 Linear Regression Model

Linear regression is one of the simplest forecasting approaches and is often used as a baseline due to its simplicity, interpretability and low computational requirements [3]. It assumes a linear relationship between variables such as irradiance and power output. While useful for capturing general trends, it struggles to model the nonlinear behavior of PV generation under changing weather conditions. Despite these limitations, they remain useful for establishing baseline performance and providing interpretability [1].

2.2 Tree Models

Tree-based algorithms, including Decision Trees, Random Forests, and Gradient Boosting, are widely used because they can model nonlinear relationships and interactions between weather variables. Random Forest improves robustness by combining multiple trees, while boosting methods enhance accuracy by correcting previous errors [5, 6]. However, these models do not inherently capture temporal dependencies and typically require feature engineering.

2.3 Deep Learning models

Deep learning architectures have become essential in forecasting due to their exceptional ability to model complex, non-linear weather relationships and sequential temporal dependencies. The most prominent models in this domain include LSTM, GRU and TCN.

LSTM networks are a type of recurrent neural network designed to learn long-term patterns through memory cells and gating mechanisms, including input, forget, and output gates [7, 8]. These mechanisms allow the model to retain relevant historical information while discarding noise, making LSTMs effective for capturing daily and seasonal PV generation patterns. However, their complex structure can lead to high computational requirements and potential overfitting when datasets are limited [9].

GRU were developed as a simpler alternative to LSTM, combining memory functions into update and reset gates [4]. This streamlined structure reduces the number of parameters, resulting in faster training while maintaining strong predictive performance. GRU are therefore particularly suitable for medium-sized PV datasets where computational efficiency is important.

TCN represent a different approach to sequence modeling by using causal and dilated convolutions to capture temporal relationships across long time horizons [10]. Unlike recurrent models, TCN enable parallel computation and stable training. While they have shown strong performance in time-series forecasting, their effectiveness depends on careful selection of hyperparameters such as kernel size and dilation factors [11].

Chapter 3

Data analysis

Three data sets were available for the project; 1) the aggregated load of all inhabitants of a house, for three different houses, 2) the production of photovoltaic pannels, 3) the weather data. The first two data sets came from the Swiss local energy community *Am Aawasser* located in Buochs (Canton of Nidwalden). The third one, was gathered by Meteo Blue Data in Horw, which is close to Aawasser. The Photovoltaic (PV) production being independent of the inhabitants consumption, the first data set was dropped.

3.1 Data sets description

3.1.1 Photovoltaic production

The PV production data set contained 35 035 entries, each corresponding to a measurement in kW and a timestamp. Measurements were taken at 15 minutes interval. The first measurement was taken the 12^{th} of December 2021 at 23:00 and the last measurement was taken the 12^{th} of December 2022 at 21:45, resulting in about a year of measurements. The distribution is right skewed (see Figure 3.1), dominated by the 0 measurements (51.15% of the dataset) and happening during the night and by bad weather.

3.1.2 Weather

The weather data set contained 8 760 entries, corresponding to a hourly measurement over a year. The first measurement was taken the 1^{st} of January 2022 at midnight and the last was taken the 31^{st} December 2022 at 23:00. Each entry covered 8 columns: temperature, sunshine duration, shortwave radiation, direct shortwave radiation, diffuse shortwave radiation, snowfall amount, relative humidity and cloud cover total. These were real-world measurements and not forecasting data. The distribution of the different features was analysed. The shortwave radiations columns had very similar distributions, as visible on Figure 3.2.

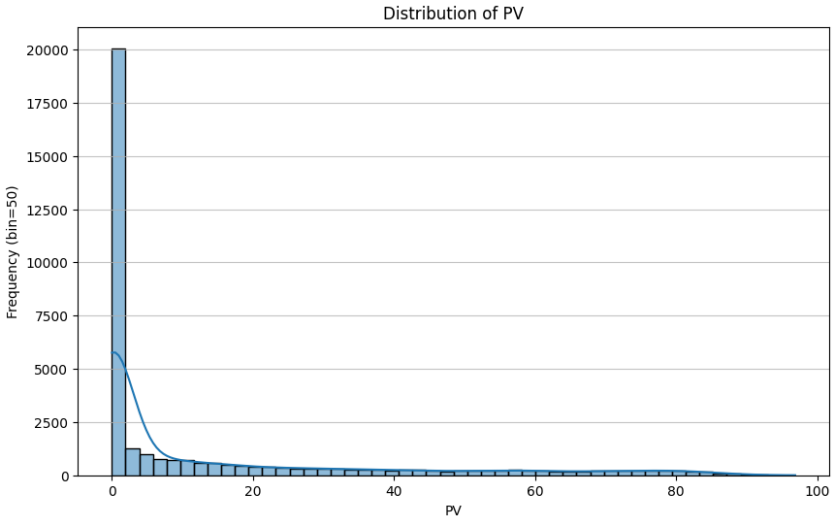

Figure 3.1: Distribution of the PV values (bin=50).

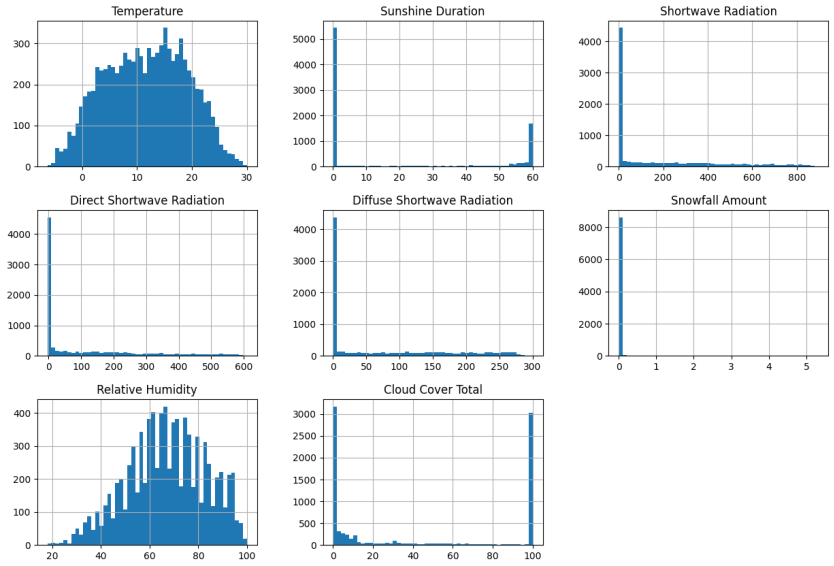

Figure 3.2: Distribution of the weather data set (bin=50).

3.2 Outliers

Different approaches were considered to tackle outliers. The first one was to interpolate all values in the 99^{th} percentile. While automatic, this method would have replaced a wide number of entries, due to the dominance of the 0 value in the PV data set. The second approach, compared the difference between each value and its predecessors and successors. If both differences surpassed an arbitrary threshold, the value would be considered as an outlier and interpolated. However, this method required to adapt the threshold for each column. Finding a meaningful threshold turned out to be challenging. Figure A.2 shows an example of the interpolation of PV outliers with the second method. After discussing with an expert, it became clear that no outlier removal was necessary, the provenance of the data set being confirmed. Extreme values and changes were measured and proofed and could be trusted. Thus, no outlier was removed.

3.3 Relationships and patterns

The seasonality of the PV data set was investigated with automated tools. No seasonality, cycle or persistence was found (see Figure A.1). This might be due to the data set covering a year only.

The relationship between the PV production and the weather data was then inspected. To do so, a Pearson correlation matrix was calculated:

$$\rho_{X,Y} = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$$

It was better suited than a regular covariance matrix, since different units were present over the columns. The matrix covered both the PV and weather data (see Figure 3.3). The target variable being the PV production, its relation with other columns was of most interest. It correlated most with the shortwave radiation (0.8), the direct shortwave radiation (0.79) and the diffuse shortwave radiation (0.78). The matrix showed these features as clear indicators of the PV. The sunshine duration correlated only at 0.61. The rest fo the feature correlated at less than 0.5. A key observation was to see the very high correlation between the three radiation measures (between 0.93 and 0.99).

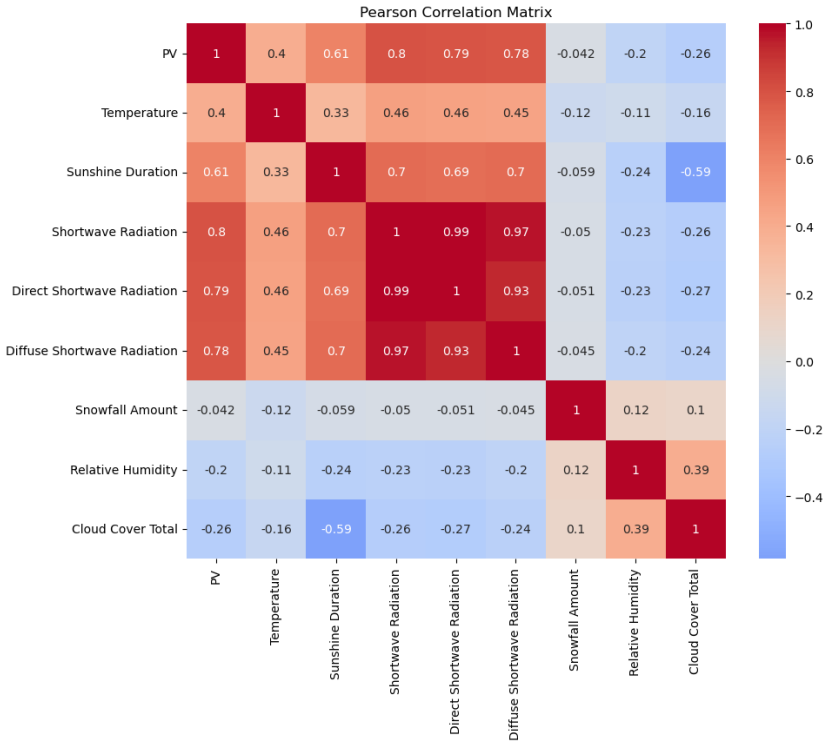

Figure 3.3: Covariance matrix between PV and weather data.

Chapter 4

Methodology

4.1 Data cleaning

Before training, several steps were applied on the data to clean it. First, a single PV value (entry 12 387) corresponding to a *Not a Number (NAN)* was replaced by interpolation. The time zone of both data sets were aligned. The timestamps in both datasets were converted to date values, then the hour and day of each observation were encoded using sine and cosine functions with the appropriate period (24 for hours, 7 for days). This circular encoding lets the model represent time-of-day and day-of-week as points on a unit circle, which is well suited for cyclical patterns. As a result, hours like 23:00 and 0:00 map to nearby positions on the circle, so the model 'sees' them as similar in time rather than as numerically far apart. Then, the resolution of the measurements differed between the PV data set and the weather one differed. Thus, the 4 PV values per hour were averaged to equal the hourly rate measurement of the weather data set. The average was chosen over the mean because it is more subject to changes and avoided flattening results. Finally, having both data frames at the same resolution, the entries were sorted per date and merged. The timestamps were not included in the training. The diffuse radiation feature was dropped. The code was created in a way to make it easy to switch between the radiation feature to drop, in order to be able to compare their impact in the future.

4.1.1 Creating data sets

Once the data frame was cleaned, it included the features mentioned in Table 4.1. The data was split into three different set: training, validation and test. The training and validation sets were used to adjust the model's weights and search different hyperparameters (see Section 4.3.3). The test set was used a single time, to assess the model's final performance. Two main criteria were considered to split the data: 1) avoid any data leakage between the sets, 2) ensure a variety of inputs over time of the day and season of the year. Thus, the data was split and sequences were created from these chunks in a second step, making impossible for a sequence to contain data from a different chunk. To ensure variety, the split was applied as follow: 2 months to test, half month to validate, half month to test (see

Final training features
PV
Temperature
Cloud cover total
Relative humidity
Sunshine duration
Snowfall amount
Hour sin
Hour cos
Day sin
Day cos
Shortwave radiation
Direct shortwave radiation

Table 4.1: Input features included in the training after data cleaning.

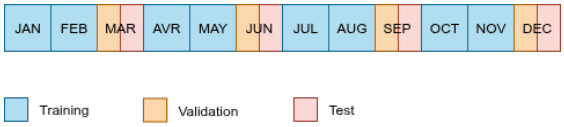

Figure 4.1: Data split into three subsets to train, validate and test the machine learning model.

Figure 4.1).

In a final step, sequences were created for each chunk separately using sliding windows. Each sequence contained 7 days of input variables with all columns but the target variable. This serves as context for the model's prediction and was chosen arbitrarily. The following day was then taken as target variable and contained only the PV values. The sequences were generated from a start every 6 hours, giving 4 sequences per day: starting from 0:00, 6:00, 12:00 and 18:00 (see Figure 4.2). In addition of multiplying the number of data, switching the starting time ensured robustness from the model. Indeed, it could otherwise predict 0 for the same indices of the sequences, corresponding to the night in most cases.

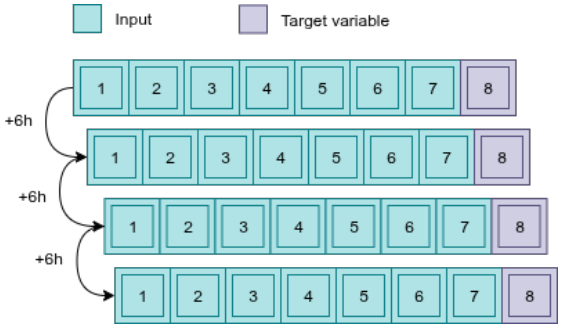

Figure 4.2: Sliding window principle used to generate sequences.

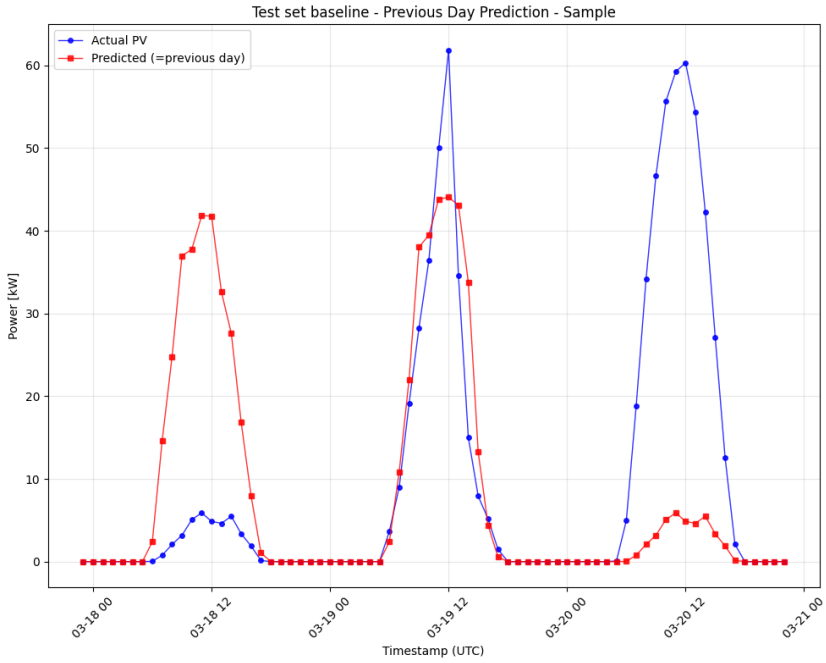

Figure 4.3: In the baseline, each day is the prediction of its successor.

4.2 Model

4.2.1 Baseline

The final model needs, given a past context information of seven days to predict the PV production for the following day, having no information about this target day at all. To assess the results, a baseline was created. It consisted of the simplest assumption possible where the next day is considered equal to the previous day in production. The mapping of a few days is visible on Figure 4.3.

4.2.2 Motivation for GRU in PV Forecasting

Short-term photovoltaic (PV) power forecasting requires models capable of jointly modeling historical generation patterns and meteorological influences. As highlighted by Wang et al. [12], PV output depends not only on current environmental conditions but also on recent production trends, making temporal dependency modeling essential.

Conventional statistical approaches often rely primarily on historical trends, while other machine learning methods incorporate meteorological variables without explicitly modeling sequential dependencies [12]. Recurrent architectures address this limitation by integrating past observations through a hidden state representation.

The GRU architecture was selected in this study due to its ability to effectively combine historical PV data and exogenous meteorological variables within a unified sequential framework [12]. Furthermore, GRU networks require fewer parameters and shorter training time compared to more complex gated architectures, while

maintaining strong predictive performance [12].

Given the sequential structure of the 168-hour input window and the requirement to learn temporal production patterns for day-ahead forecasting, a GRU-based model represented a suitable and computationally efficient choice.

4.2.3 Proposed GRU Architecture

Input and Output Definition

The forecasting task was formulated as a supervised learning problem with a fixed input window and forecast horizon. For each training sample, the model received the previous 168 hours (7 days) of historical information:

- historical PV power production,
- hourly meteorological variables,
- engineered cyclical time features.

The input can be written as a sequence $\mathbf{X} \in \mathbb{R}^{168\times F}$, where F denotes the number of input features. The target is a 24-hour day-ahead PV forecast $\mathbf{y} \in \mathbb{R}^{24}$, corresponding to hours $t+1$ to $t+24$.

Network Architecture

The proposed forecasting model consisted of a single-layer Gated Recurrent Unit (GRU) network followed by a fully connected output layer.

The input to the model was defined as a tensor

$$\mathbf{X} \in \mathbb{R}^{T\times F},$$

where $T = 168$ represented the past 168 hourly observations (corresponding to seven days of historical data), and F denoted the number of input features. These features included historical PV production, meteorological variables, and engineered cyclical time-related features.

The GRU processed the input sequence sequentially and produced a hidden state

$$\mathbf{h}_t \in \mathbb{R}^{64}$$

at each time step $t \in \{1, \ldots, T\}$. The hidden size was fixed to 64 units, allowing the network to encode temporal dependencies in a compact latent representation.

Only the final hidden state,

$$\mathbf{h}_T,$$

was retained and used as a summary representation of the entire 168-hour input sequence. This final hidden state was interpreted as a compressed encoding of the recent temporal dynamics of PV production and associated exogenous variables.

Subsequently, the representation $\mathbf{h}_T$ was passed to a fully connected linear layer,

$$\hat{\mathbf{y}} = \mathbf{W}\mathbf{h}_T + \mathbf{b},$$

where $\mathbf{W} \in \mathbb{R}^{24\times64}$ and $\mathbf{b} \in \mathbb{R}^{24}$. This layer projected the 64-dimensional hidden representation to a 24-dimensional output vector corresponding to the 24-hour day-ahead PV forecast.

4.3 Training

4.3.1 Feature Scaling

All input features were standardised using a z-score normalisation,

$$x' = \frac{x - \mu}{\sigma},$$

where μ and σ were estimated exclusively from the training set. The same transformation was then applied to the validation and test sets to avoid data leakage.

4.3.2 Loss Function

Model training was performed using the Huber loss (also referred to as SmoothL1 loss), which combines properties of both squared error and absolute error loss [13].

The Huber loss with tuning parameter $\beta > 0$ is defined as

$$L_\beta(e) = \begin{cases} \frac{1}{2}e^2 & \text{if } |e| \le \beta, \\ \beta|e| - \frac{1}{2}\beta^2 & \text{otherwise,} \end{cases}$$

where $e = \hat{y} - y$ denotes the forecast error.

For small deviations, the loss behaves quadratically, similar to mean squared error. For large deviations, it transitions to a linear penalty, similar to mean absolute error. This formulation provides robustness against outliers while maintaining smooth gradient behaviour near zero error [13].

The tuning parameter β controls the transition point between quadratic and linear penalisation.

4.3.3 Optimisation and Regularisation

Model parameters were optimised using the Adam algorithm [14]. Adam is an adaptive first-order optimisation method that maintains exponentially decaying averages of both past gradients (first moment) and squared gradients (second moment), enabling parameter-wise adaptive learning rates. This allows faster and more stable convergence compared to standard stochastic gradient descent [14].

Weight decay (L2 regularisation) was optionally applied to penalise large model parameters.

The learning rate was dynamically adjusted using a validation-based scheduler (ReduceLROnPlateau[1]), which decreases the learning rate when the validation loss stops improving. This improves convergence stability in later training stages [14].

[1] `https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.ReduceLROnPlateau.html`

To further stabilise optimisation, gradient clipping was applied by constraining the ℓ_2-norm of the gradient to a maximum value of 1.0. This prevents excessively large parameter updates and mitigates exploding gradients in recurrent networks.

Training was performed for up to 400 epochs. Early stopping was applied based on the validation loss to prevent overfitting and to select the model with the best generalisation performance.

Table 4.2: Training configuration used for all runs.

Component	Configuration
Optimiser	Adam
Loss	Huber (Smooth L1), $\delta \in \{0.5, 1.0\}$ [?]
Batch size	64
Max. epochs	400
Early stopping	patience = 10, min_delta = 10^{-5}
LR scheduler	ReduceLROnPlateau (factor = 0.5, patience = 3, min_lr = 10^{-6})
Gradient clipping	$\|\nabla\|_2 \leq 1.0$
Model selection	best validation loss

4.3.4 Evaluation Metrics

Forecasting performance was evaluated using standard regression-based error metrics commonly employed in time series forecasting research [15]. Specifically, Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE) were computed. A description of these metrics and their formulas are available in Appendix B.

MAPE expresses prediction error in relative percentage terms. However, in photovoltaic power forecasting, PV production frequently equals zero during night hours. Since MAPE involves division by the true value y_t, zero or near-zero production values lead to extremely large or undefined percentage errors. In the present dataset, this resulted in large MAPE values that do not reflect the actual forecasting quality during daylight hours.

4.3.5 Hyperparameter Search

Hyperparameters are configuration variables that govern the learning process and model structure and are defined prior to training [16].

In this work, hyperparameter optimisation was performed using grid search. Grid search evaluates all predefined combinations within a specified search space and is considered a straightforward and systematic approach to hyperparameter optimisation [16].

The following hyperparameters were tuned:

- **Learning rate**: controls the step size during gradient-based optimisation and directly affects convergence speed and stability.

- **Hidden size**: determines the representational capacity of the recurrent layer.
- **Number of GRU layers**: influences model depth and temporal abstraction capability. While it affects the architecture of the model and not only its parameters, the GRU layers were included in the automated search for convenience.
- **Dropout rate**: acts as a regularisation mechanism to reduce overfitting.
- **Weight decay**: applies L2 regularisation to discourage excessively large weights.
- **Huber parameter** δ: controls the transition point between quadratic and linear penalisation in the loss function.

The following hyperparameters and their values were used:

Table 4.3: Hyperparameter search space.

Hyperparameter	Values
Learning rate	$\{10^{-3}, 5 \times 10^{-4}, 10^{-4}\}$
Hidden size	$\{32, 64, 128\}$
Number of layers	$\{1, 2\}$
Dropout	$\{0.0, 0.1, 0.2\}$
Weight decay	$\{0, 10^{-4}\}$
Huber δ	$\{0.5, 1.0\}$

Although the full grid would yield 216 configurations, combinations with a single GRU layer and non-zero dropout were excluded, as dropout is only applied between stacked recurrent layers in PyTorch. This resulted in a total of 144 evaluated configurations.

Chapter 5

Results

5.1 Overall Performance

Table 5.1: Test set forecasting performance comparison.

Metric	Baseline	GRU
MAE (kW)	5.27	6.87
RMSE (kW)	13.31	11.80

Table 5.1 summarises the overall forecasting performance. While the GRU achieved a comparable RMSE, it exhibited a higher MAE than the baseline, indicating weaker average absolute performance. Training and validation learning curves confirmed stable convergence without strong overfitting and are provided in Appendix C.

5.2 Error analysis

Beyond aggregate performance metrics, a comprehensive error analysis was conducted to better understand the behaviour of the GRU model. The following aspects were investigated: Overall performance comparison with the persistence baseline, average 24-hour profile behaviour, systematic prediction bias (prediction vs. true analysis), horizon-dependent error variation, sample-wise error variability, day versus night error distribution, absolute error distribution characteristics.

5.2.1 Average 24h PV Curve

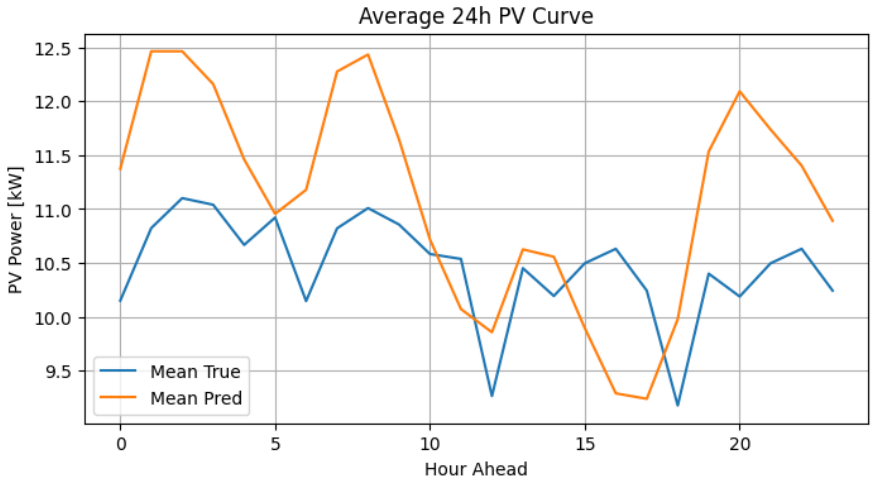

Figure 5.1: Error Analysis: Average 24h PV curve.

Figure 5.1 presents the average 24-hour PV profile across the test set. The model captures the general daily pattern but introduces systematic horizon-dependent bias. Early and late forecast hours are overestimated, whereas peak production around midday is consistently underestimated.

5.2.2 Prediction vs. True Scatter Plot

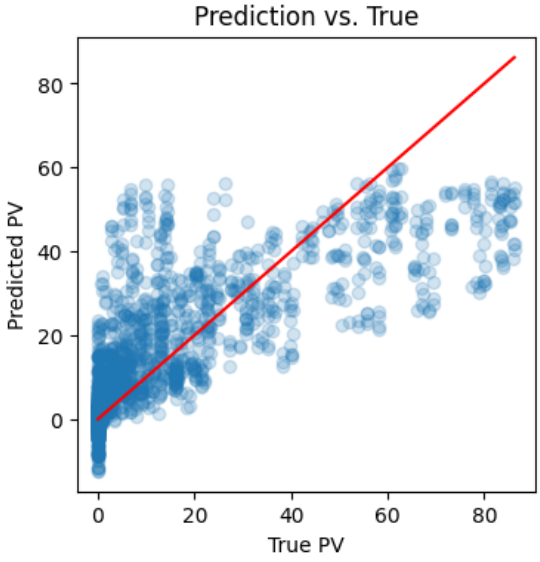

Figure 5.2: Error Analysis: Predictions vs. True Values.

Figure 5.2 shows predicted versus true PV values across all forecast horizons. High production values are systematically underestimated, while low values are slightly overestimated, indicating conservative regression-to-the-mean behaviour.

5.2.3 MAE per Sample

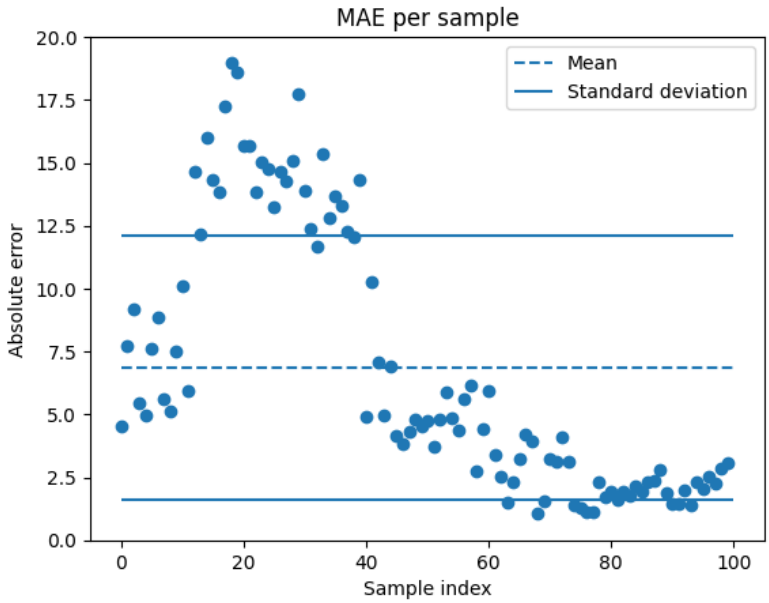

Figure 5.3: Error Analysis: MAE per Sample.

Figure 5.3 presents the MAE for each individual test sample. The distribution reveals variability in forecasting accuracy across different days. While many samples exhibit relatively low error levels, a subset of days shows significantly increased MAE, indicating heterogeneous model performance.

The presence of high-error samples suggests that the GRU struggles particularly during complex or highly variable PV patterns.

Chapter 6

Conclusion

Although the GRU was able to capture the general daily production pattern, the persistence baseline achieved better overall MAE performance. Error analysis revealed that the GRU systematically underestimated peak production and exhibited regression-to-the-mean behaviour. Performance varied substantially across individual days, indicating sensitivity to complex or rapidly changing weather conditions. These findings suggest that, for the investigated dataset, increased model complexity did not automatically translate into improved forecasting accuracy. Future work could explore shorter context windows. Incorporating explicit weather forecast data for the prediction day may further improve performance by providing forward-looking information. Additionally, combining deep learning approaches with statistical uncertainty modeling may enhance robustness and interpretability.

6.1 Lessons learned

This project was our first in the forecasting field. To approach it, we tried to benefit from our complementing backgrounds in Artificial Intelligence and Machine Learning and Energy and Environmental Systems Engineering. While we all exchanged and supported each other, one person took more care of the state of the art, one of us focused on the data analysis, one built the architecture model. Interacting with the teachers and the other teams was beneficial, and showed the variety of approaches possible to solve the same problem. This project introduced us to the forecasting field, including time series. Since it was a first, anticipation and planning appeared difficult and we noticed our mistakes only after making them. Sharing code and files became challenging and some issues were experienced with Google Colab. For the next project, a proper environment would be setup, for example with GitHub, and more time would be spent on the data analysis. Overall, this week was a very positive learning experience.

Bibliography

[1] J. Kim, D. Kim, W. Yoo, J. Lee, and Y. B. Kim, "Daily prediction of solar power generation based on weather forecast information in Korea," *IET Renewable Power Generation*, vol. 11, no. 10, pp. 1268–1273, 2017. DOI: 10.1049/iet-rpg.2016.0698.

[2] Y. Essam, A. N. Ahmed, R. Ramli, K.-W. Chau, M. S. Idris Ibrahim, M. Sherif, A. Sefelnasr, and A. El-Shafie, "Investigating photovoltaic solar power output forecasting using machine learning algorithms," *Engineering Applications of Computational Fluid Mechanics*, vol. 16, no. 1, pp. 2002–2034, 2022. DOI: 10.1080/19942060.2022.2126528.

[3] C. Voyant, G. Notton, S. Kalogirou, M.-L. Nivet, C. Paoli, F. Motte, and A. Fouilloy, "Machine learning methods for solar radiation forecasting: A review," *Renewable Energy*, vol. 105, pp. 569–582, 2017. DOI: 10.1016/j.renene.2016.12.095.

[4] M. T. Sattari, H. Apaydin, and S. Shamshirband, "Performance evaluation of deep learning-based Gated Recurrent Units (GRUs) and tree-based models for estimating ETo by using limited meteorological variables," *Mathematics*, vol. 8, no. 6, p. 972, 2020. DOI: 10.3390/math8060972.

[5] B. Carrera and K. Kim, "Comparison analysis of Machine Learning techniques for Photovoltaic prediction using Weather Sensor Data," *Sensors*, vol. 20, no. 11, p. 3129, 2020. DOI: 10.3390/s20113129.

[6] M. Meng and C. Song, "Daily Photovoltaic Power Generation forecasting model based on Random Forest algorithm for North China in Winter," *Sustainability*, vol. 12, no. 6, p. 2247, 2020. DOI: 10.3390/su12062247.

[7] M. AlKandari and I. Ahmad, "Solar power generation forecasting using ensemble approach based on deep learning and statistical methods," *Applied Computing and Informatics*, vol. 20, no. 3/4, pp. 231–250, 2024.

[8] N. Zhou, Y. Zhou, L. Gong, and M. Jiang, "Accurate prediction of photovoltaic power output based on long short-term memory network," *IET Optoelectronics*, vol. 14, no. 6, pp. 399–405, 2020.

[9] F. Harrou, F. Kadri, and Y. Sun, "Forecasting of Photovoltaic Solar Power Production using LSTM approach," in *Advanced Statistical Modeling, Forecasting, and Fault Detection in Renewable Energy Systems* (F. Harrou and Y. Sun, eds.), ch. 2, IntechOpen, 2020. DOI: 10.5772/intechopen.91248.

[10] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018. DOI: 10.48550/arXiv.1803.01271.

[11] C. Lea, M. D. Flynn, R. Vidal, A. Reiter, and G. D. Hager, "Temporal Convolutional Networks for Action Segmentation and Detection," *arXiv preprint arXiv:1611.05267*, 2016. DOI: 10.48550/arXiv.1611.05267.

[12] Y. Wang, W. Liao, and Y. Chang, "Gated Recurrent Unit Network-Based Short-Term Photovoltaic Forecasting," *Energies*, vol. 11, no. 8, p. 2163, 2018. DOI: 10.3390/en11082163.

[13] R. J. Taggart, "Point Forecasting and Forecast Evaluation with Generalized Huber Loss," *Electronic Journal of Statistics*, vol. 16, pp. 201–231, 2022. DOI: 10.1214/21-EJS1957.

[14] D. Zou, Y. Cao, Y. Li, and Q. Gu, "Understanding the Generalization of Adam in Learning Neural Networks with Proper Regularization," 2021. DOI: 10.48550/arXiv.2108.11371.

[15] G. Lu, O. Yang, Z. Wang, Y. Qu, Y. Xia, D. Tang, I. Kotenko, and W. Li, "A Survey of Deep Learning for Time Series Forecasting: Theories, Datasets, and State-of-the-Art Techniques," *Computers, Materials, & Continua*, vol. 85, no. 2, pp. 2403–2441, 2025. DOI: 10.32604/cmc.2025.059431.

[16] J. A. Ilemobayo *et al.*, "Hyperparameter Tuning in Machine Learning: A Comprehensive Review," *Journal of Engineering Research and Reports*, vol. 26, no. 6, pp. 388–395, 2024. DOI: 10.9734/jerr/2024/v26i61188.

Appendix A

Graphs

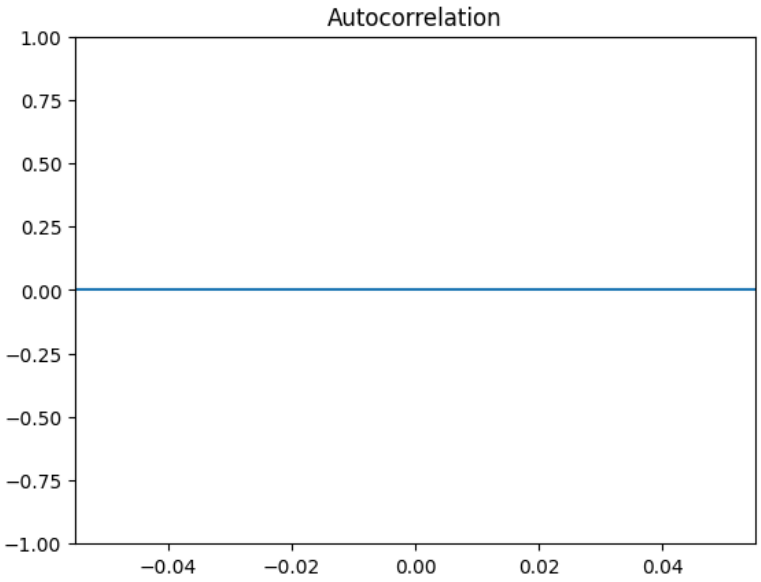

Figure A.1: Autocorrelation of PV production.

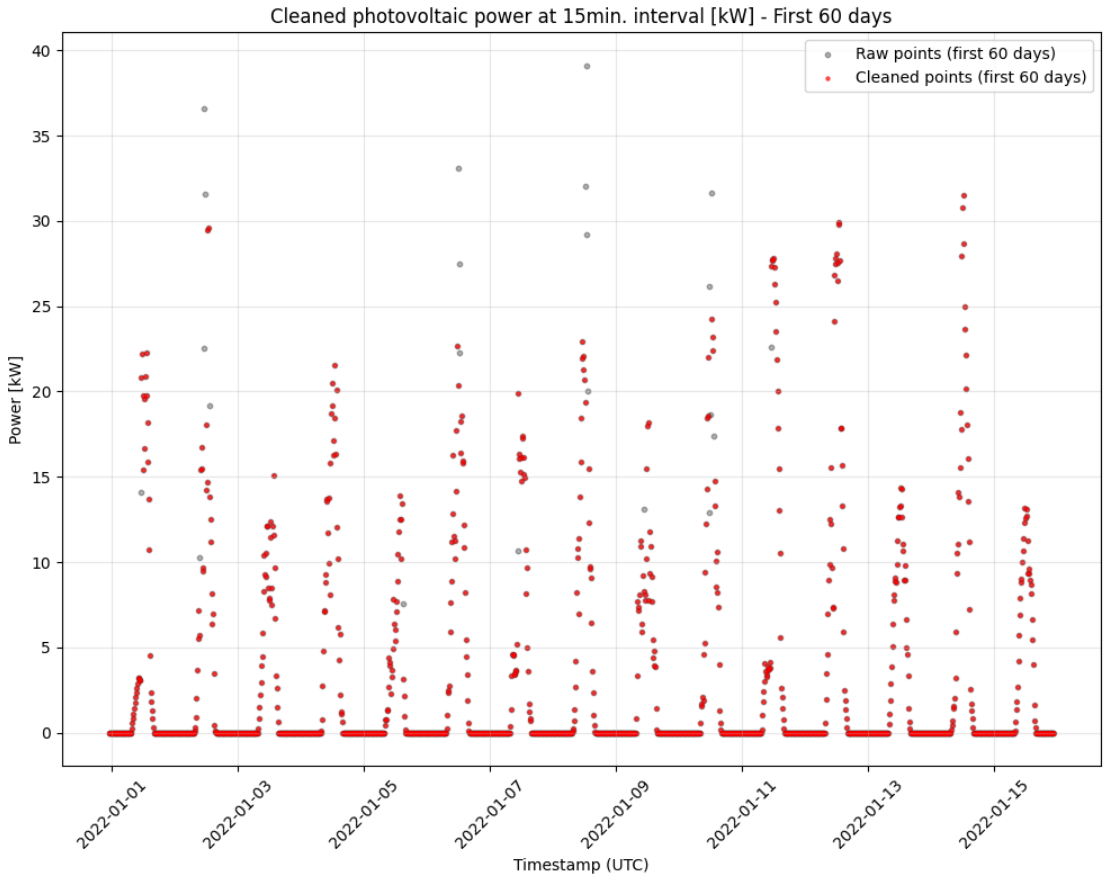

Figure A.2: Data entries before and after interpolating outliers using the second approach with a threshold of 5, on a sample of 60 days.

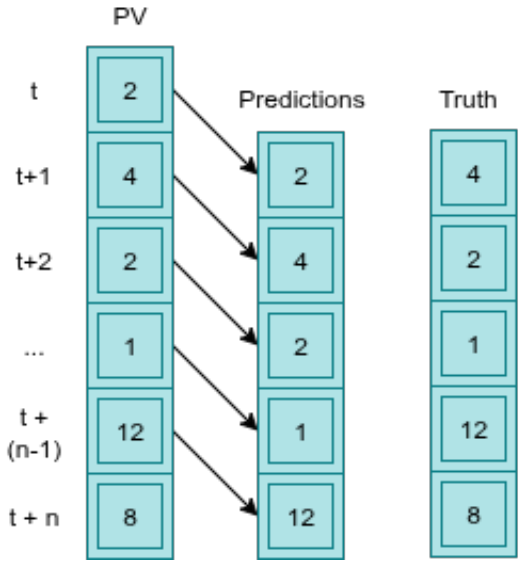

Figure A.3: Illustration of the baseline generation.

Appendix B

Evaluation metrics formulas

Let y_t denote the true PV value and $\hat{y}_t$ the predicted value at time step t, for $t = 1, \ldots, N$.

Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{N} \sum_{t=1}^{N} |y_t - \hat{y}_t|$$

MAE measures the average magnitude of prediction errors in the same unit as the target variable (kW), providing an intuitive measure of typical deviation.

Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{N} \sum_{t=1}^{N} (y_t - \hat{y}_t)^2$$

MSE penalises larger errors more strongly due to quadratic scaling, making it sensitive to peak forecasting errors.

Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

RMSE expresses the squared error in the original unit (kW), improving interpretability while retaining sensitivity to large deviations.

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE} = \frac{100}{N} \sum_{t=1}^{N} \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

Appendix C

Learning Curve

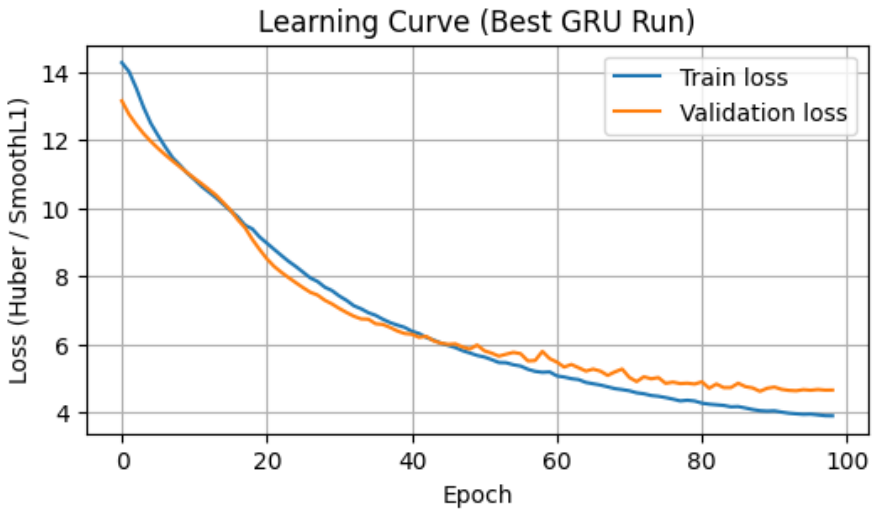

Figure C.1: Illustration of the Learning Curve (Training Loss and Validation Loss).

Appendix D

Usage of LLM

Intended use	**Description of the specific use**	**Area**	**AI tool(s) used**
Transform wording	LLMs were used to rewrite sections Results & Error analysis	Language	ChatGPT `https://chatgpt.com`; Perplexity `https://www.perplexity.ai/`
Save files	Help with saving training files to the drive, delivered the code to save all important training information	Coding	ChatGPT `https://chatgpt.com`

Table D.2: Usage of AI tools